
raft-rx — User guide: Python

FSM-based Raft implementation over rxnet

2026-05-10

Contents

raft_rx - Python User Guide	5
Part I - The Library	7
1. Overview	7
2. Architecture	8
3. Key Concepts	8
Roles	8
Log, Terms and Commit	9
Membership Configuration	9
4. Implementing an Application	9
Example: Key-Value Application	10
5. Building a Cluster	11
Running Ticks	12
6. Submitting Commands	12
7. Membership Changes	13
8. Storage Format	13
meta.json	14
log.json	14
snapshot.json	14
9. Observability and Tracing	15
JSONL Telemetry	15
rxnet Trace	15
10. Shell	15
Generic Commands	16
KV Commands	16
Part II - The Demo Application	19
11. Overview	19
What the demo app adds on top of raft-rx	19
Source layout	20
12. Running the Demo	20

13. Restarting a Node	21
14. Adding and Removing Nodes	22
Adding a node at startup	22
Adding a node from the CLI	22
Removing a node	22
15. Joining and Merging Running Clusters	22
16. CLI Reference	23
Key-value commands	23
Cluster inspection	23
Cluster management	24
17. REST API	24
Client and inspection endpoints	24
Raft transport endpoints	25
18. Demo Troubleshooting	26
The joining node stays alone	26
Writes work on one node but reads look stale elsewhere	26
A restarted node has unexpected old data	26
Part III - Reference	27
19. Public API Reference	27
Package Exports	27
NodeConfig	27
RaftCluster	28
RaftNode	28
20. Building and Testing	29
21. Troubleshooting	29
No leader is elected	29
Commands are not committed	30
A joining node does not participate	30
Snapshot restore misses state	30

List of Figures

raft_rx - Python User Guide

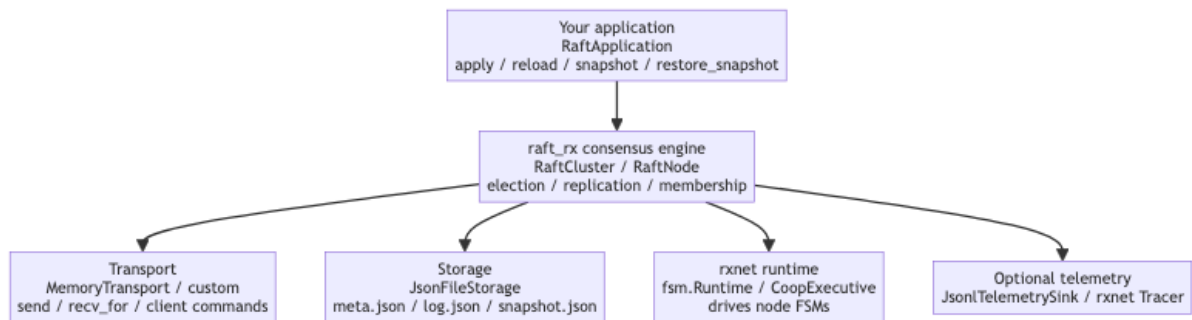
Part I - The Library

1. Overview

`raft_rx` is a Python package that implements the Raft consensus algorithm on top of the `rxnet` FSM runtime. It provides:

- **Replicated state machine** - commands are appended to a distributed log, committed by majority quorum, and applied to your application in order.
- **Leader election** - deterministic tests via `ManualClock`, or wall-clock operation via `SystemClock`.
- **Membership changes** - joint-consensus reconfiguration through `request_membership_change`.
- **Persistent storage** - term, vote, log, snapshot and membership configuration are stored as JSON files.
- **Pluggable transport** - the package includes deterministic in-memory transport; the standalone demo shows an HTTP transport.
- **Pluggable application** - implement a small protocol to attach your own state machine.
- **Optional observability** - JSONL telemetry and `rxnet` trace export can be enabled per cluster.

2. Architecture



Layer	Type	Responsibility
Application	<code>RaftApplication</code>	Your state machine: what commands mean
Consensus engine	<code>RaftCluster</code> / <code>RaftNode</code>	Raft protocol, log replication and membership
Transport	<code>MemoryTransport</code> or compatible object	Deliver Raft messages and client commands
Storage	<code>JsonFileStorage</code>	Durable protocol state and snapshots
Runtime	<code>rxnet.fsm.Runtime</code>	Ticks the Raft FSMs
Telemetry	<code>JsonTelemetrySink</code> / <code>NullTelemetrySink</code>	Optional structured events

3. Key Concepts

Roles

Every running node is in one of three roles:

Role	Description
<code>FOLLOWER</code>	Accepts log entries and votes in elections
<code>CANDIDATE</code>	Requests votes after its election timeout expires
<code>LEADER</code>	Accepts client commands, replicates and commits them

Log, Terms and Commit

- Each client command becomes a `LogEntry(index, term, command, leader_id)`.
- The leader replicates entries to a quorum, then advances `commit_index`.
- Committed entries are applied through your application's `apply` method.
- `term` is a monotonically increasing election epoch.

Membership Configuration

Membership is represented by `ClusterConfiguration`:

- `old_members`: current stable set, or leaving set during joint consensus.
- `new_members`: joining set during joint consensus, otherwise `None`.
- `index`: log index where the configuration was written.

`mode()` returns `"stable"` or `"joint"`. During joint consensus, commit requires a majority in both `old_members` and `new_members`.

4. Implementing an Application

Your application must implement the `RaftApplication` protocol:

```
1 from raft_rx import Command
2
3
4 class RaftApplication:
5     def apply(self, command: Command) -> None: ...
6     def reload(self) -> None: ...
7     def snapshot(self) -> object: ...
8     def restore_snapshot(self, snapshot: object) -> None: ...
```

Commands are simple dataclasses:

```
1 from raft_rx import Command
2
3 Command(op="set", key="color", value="blue")
4 Command(op="delete", key="color")
```

Method	When it is called	What it should do
<code>apply</code>	Once per committed log entry, in index order	Mutate application state
<code>reload</code>	Restart with no snapshot	Reset or reload the empty/base state
<code>snapshot</code>	Log compaction	Return a JSON-serializable snapshot object
<code>restore_snapshot</code>	Restart from snapshot	Restore state from the snapshot object

Correctness rule: mutate replicated state only from `apply`. Reads can inspect the local applied state, but writes must enter the leader's log.

Example: Key-Value Application

```
1 import json
2 from pathlib import Path
3
4 from raft_rx import Command
5
6
7 class KVApp:
8     def __init__(self, path: Path) -> None:
9         self.path = path
10        self.path.parent.mkdir(parents=True, exist_ok=True)
11        self.data: dict[str, str] = {}
12        self.reload()
13
14    def reload(self) -> None:
15        if self.path.exists():
16            self.data = {
17                str(k): str(v)
18                for k, v in json.loads(self.path.read_text()).items()
19            }
```

```

20         else:
21             self.data = {}
22
23     def apply(self, command: Command) -> None:
24         if command.op == "set":
25             if command.value is None:
26                 raise ValueError("set requires value")
27             self.data[command.key] = command.value
28         elif command.op == "delete":
29             self.data.pop(command.key, None)
30         else:
31             raise ValueError(f"unsupported op: {command.op}")
32         self.path.write_text(json.dumps(self.data, indent=2, sort_keys=
33                                 True))
34
35     def get(self, key: str) -> str | None:
36         return self.data.get(key)
37
38     def snapshot(self) -> object:
39         return dict(self.data)
40
41     def restore_snapshot(self, snapshot: object) -> None:
42         self.data = {str(k): str(v) for k, v in dict(snapshot).items()}
43         self.path.write_text(json.dumps(self.data, indent=2, sort_keys=
44                                 True))

```

The repository version in `python/examples/kv_app.py` also writes via a temporary file and atomic replace.

5. Building a Cluster

Use `RaftCluster` for local deterministic simulations and tests.

```

1  from pathlib import Path
2
3  from raft_rx import ManualClock, NodeConfig, RaftCluster
4  from raft_rx.cluster import ClusterNodePaths
5
6
7  root = Path("var/my-cluster")
8  cluster = RaftCluster(clock=ManualClock())
9  node_ids = ["n1", "n2", "n3"]
10
11  for i, node_id in enumerate(node_ids):
12      peers = [peer for peer in node_ids if peer != node_id]

```

```

13     cluster.add_node(
14         NodeConfig(
15             node_id=node_id,
16             peers=peers,
17             election_timeout_ms=150 + i * 100,
18             heartbeat_interval_ms=50,
19         ),
20         ClusterNodePaths(
21             storage_dir=root / "storage",
22             telemetry_path=root / "telemetry" / f"{node_id}.jsonl",
23         ),
24         application=KVApp(root / "kv" / f"{node_id}.json"),
25     )

```

`NodeConfig.initial_members` is optional. If omitted, the node starts with `[node_id, *peers]`. Use an explicit list when you need all nodes to share a particular initial ordering.

Running Ticks

With `ManualClock`, each tick advances time explicitly:

```

1 cluster.run(40, advance_ms=10)
2 leader = cluster.leader()

```

With `SystemClock`, use an `rxnet` executive, as shown by the standalone demo:

```

1 from rxnet.coop import CoopExecutive
2
3 executive = CoopExecutive()
4 executive.add(cluster.runtime)
5 executive.run()

```

6. Submitting Commands

Commands must reach the leader to be committed.

```

1 from raft_rx import Command
2
3 leader = cluster.leader()
4 if leader is None:
5     raise RuntimeError("no leader")
6
7 leader.submit_command(Command(op="set", key="color", value="blue"))

```

```
8 cluster.run(30, advance_ms=10)
```

`RaftCluster.submit_to_leader` is a convenience wrapper:

```
1 ok = cluster.submit_to_leader(Command(op="delete", key="color"))
```

If a command is submitted to a follower in the core in-memory transport, that follower drains and rejects it. Production-style forwarding belongs in the application or transport layer; the standalone HTTP demo implements that.

7. Membership Changes

Call `request_membership_change` on the current leader with the full target member list, not a delta.

```
1 leader = cluster.leader()
2 if leader is None:
3     raise RuntimeError("no leader")
4
5 cluster.provision_node("n4", members=["n1", "n2", "n3", "n4"], running=
    True)
6 leader.request_membership_change(["n1", "n2", "n3", "n4"])
7 cluster.run(120, advance_ms=10)
```

To remove a node, omit it from the target list:

```
1 leader.request_membership_change(["n1", "n2", "n3"])
```

Internally the membership FSM writes:

1. `cluster.enter_joint`
2. `cluster.leave_joint`

During the joint phase, quorum requires a majority from both old and new configurations.

8. Storage Format

`JsonFileStorage(root, node_id)` stores files under `{root}/{node_id}/`:

```
1 var/my-cluster/storage/└─
2   n1/└─
3     meta.json└─
4     log.json└─
5     snapshot.json
```

meta.json

Contains protocol metadata:

```
1 {
2   "commit_index": 2,
3   "compaction_threshold": 32,
4   "configuration_index": 0,
5   "current_term": 1,
6   "new_members": null,
7   "old_members": ["n1", "n2", "n3"],
8   "snapshot_last_included_index": 0,
9   "snapshot_last_included_term": 0,
10  "voted_for": "n1"
11 }
```

log.json

Contains un-compacted log entries:

```
1 [
2   {
3     "index": 1,
4     "term": 1,
5     "leader_id": "n1",
6     "command": {"op": "set", "key": "color", "value": "blue"}
7   }
8 ]
```

snapshot.json

Contains compaction metadata plus your application snapshot:

```
1 {
2   "application": {"color": "blue"},
3   "last_included_index": 32,
4   "last_included_term": 4
5 }
```

`JsonFileStorage` writes JSON via temp file plus replace.

9. Observability and Tracing

JSONL Telemetry

Pass a telemetry path when adding each node:

```
1 ClusterNodePaths(  
2     storage_dir=root / "storage",  
3     telemetry_path=root / "telemetry" / f"{node_id}.jsonl",  
4 )
```

Each line is a JSON object with time, role, term, event and payload. The shell command `events [NODE ...]` displays the last events.

rxnet Trace

Enable tracing on the cluster:

```
1 cluster = RaftCluster(clock=ManualClock(), trace_enabled=True)
```

Export the trace:

```
1 cluster.export_trace(root / "trace.bin")  
2 cluster.report_trace(root / "trace.html")
```

The example `python/examples/kv_reconfigure_trace.py` generates:

- `python/var/python-reconfigure-trace/trace.bin`
 - `python/var/python-reconfigure-trace/trace.html`
-

10. Shell

Run the generic shell:

```
1 uv run --project python python/tools/raftsh.py
```


Run the KV shell:

```
1 uv run --project python python/examples/kv_shell.py
```

Generic Commands

Command	Description
<code>status</code>	Cluster status and per-node summary
<code>leader</code>	Current leader
<code>tick [N] [MS]</code>	Advance the simulation
<code>autotick [MS]</code>	Configure background ticking; 0 disables it
<code>stop NODE</code>	Stop a node
<code>start NODE</code>	Start a node
<code>restart NODE</code>	Stop and start a node
<code>members [NODE]</code>	Show stable or joint membership
<code>addnode NODE</code>	Provision and add a node via joint consensus
<code>rmnode NODE</code>	Remove a node via joint consensus
<code>log NODE</code>	Print local log entries and commit status
<code>maxlog [ENTRIES]</code>	Read or update compaction threshold
<code>dump</code>	Print summaries as JSON
<code>trace [PATH]</code>	Export <code>trace.bin</code> if tracing is enabled
<code>trace_report [PATH]</code>	Export an HTML trace report
<code>events [NODE ...]</code>	Show recent JSONL telemetry
<code>quit/exit</code>	Leave the shell

KV Commands

The KV shell registers additional commands:

Command	Description
<code>set KEY VALUE</code>	Submit a replicated write to the leader
<code>get KEY [NODE]</code>	Read local applied state from a node
<code>delete KEY</code>	Submit a replicated delete to the leader

Part II - The Demo Application

11. Overview

`python/examples/demo_app` is a separate application that consumes `raft-rx` as an external dependency. It is the best place to study how the library is used in a real process boundary: each process hosts one Raft node, exposes an HTTP transport, runs a local interactive CLI, persists its own state, and uses `HOST:PORT` as the Raft node ID.

It is **one possible application** of the library, not the library itself. Read Part I to understand the core API; read this part to run and inspect a complete multi-process key-value service.

What the demo app adds on top of raft-rx

Concern	How the demo app handles it
State machine	<code>DemoKVApp</code> , a persisted key-value application
Transport	HTTP transport implemented with the Python standard library
Executor	<code>rxnet.coop.CoopExecutive</code> driving the Raft node and CLI FSM
Command routing	Followers forward writes to the known leader
Node identity	The advertised <code>HOST:PORT</code> string is also the Raft node ID
API surface	Local CLI plus REST endpoints for clients and peer nodes

Source layout

1	python/examples/demo_app/	—
2	pyproject.toml	—
3	README.md	—
4	src/raft_rx_demo/	—
5	demo.py	process wiring, HTTP server, runtime and REST API —
6	app.py	replicated key-value application —
7	transport.py	HTTP transport and REST client helpers —
8	cli.py	interactive CLI FSM

The demo intentionally keeps transport, application logic and CLI wiring in separate modules. That makes it easier to replace one concern at a time in your own application.

12. Running the Demo

Run commands in this section from the `python/` directory.

Start the first node:

```
1 uv run --project examples/demo_app raft-rx-demo --bind 127.0.0.1:7400
```

Start a second node and ask it to join the first:

```
1 uv run --project examples/demo_app raft-rx-demo \
2   --join 127.0.0.1:7400 \
3   --bind 127.0.0.1:7401
```

Start a third node the same way:

```
1 uv run --project examples/demo_app raft-rx-demo \
2   --join 127.0.0.1:7400 \
3   --bind 127.0.0.1:7402
```

Each process opens an interactive prompt. After election, one node becomes leader. You can check from any prompt:

```
1 status
2 members
```

Submit a write from any node:

```
1 set color blue
```

If the local node is a follower, the demo forwards the write to the leader. Reads are local:

```
1 get color
```

A local read returns the state already applied on that process. In a healthy cluster it quickly converges on all nodes, but it is not a linearizable read protocol by itself.

Useful flags:

Flag	Description
<code>--bind HOST:PORT</code>	Address to bind and advertise as node ID
<code>--join HOST:PORT</code>	Existing node to join at startup
<code>--data-dir DIR</code>	Persistent state directory
<code>--tick-ms MS</code>	Runtime tick period; default 25

By default, data is stored under `var/demo/<host>_<port>`. Use `--data-dir` when you want predictable paths for examples or tests.

13. Restarting a Node

Stop one process with `quit`, then restart it with the same bind address and data directory:

```
1 uv run --project examples/demo_app raft-rx-demo --bind 127.0.0.1:7401
```

The node reloads its durable Raft state and local KV snapshot from disk. It does not need to be re-added if it was still a member of the cluster when it stopped; the member IDs are `HOST:PORT` addresses, so the HTTP transport can contact peers directly from the persisted membership configuration.

A three-node cluster remains available while one node is offline. If the leader is stopped, the remaining voters elect a new leader after the election timeout.

14. Adding and Removing Nodes

Adding a node at startup

The simplest way to add a new process is `--join`:

```
1 uv run --project examples/demo_app raft-rx-demo \  
2   --join 127.0.0.1:7400 \  
3   --bind 127.0.0.1:7403
```

The introducer forwards the join request through the cluster, peers learn the new address, and the leader submits a membership change. The new node becomes a voting member once the joint-consensus transition finishes.

Adding a node from the CLI

If the target process is already running, use `addnode` from any prompt:

```
1 addnode 127.0.0.1:7403
```

The command is forwarded to the leader if necessary.

Removing a node

Remove a node by passing the node ID, which is its `HOST:PORT` string:

```
1 rmnode 127.0.0.1:7403
```

The demo sends a full target membership list to `request_membership_change`. Internally the Raft engine appends `cluster.enter_joint` and `cluster.leave_joint` entries, so the same safety rules described in Part I apply.

15. Joining and Merging Running Clusters

`join HOST:PORT` makes the current node join the cluster reachable at the target address:

```
1 join 127.0.0.1:7400
```

Use `join` when a single process should become a member of an existing cluster.

`merge HOST:PORT` is different: it combines two independent clusters. Run it from any node in cluster A and point it at any node in cluster B:

```
1 merge 127.0.0.1:7500
```

The two sides exchange their known members and addresses, then each leader requests the union of both member sets. Use `merge` for demos where two clusters were started independently and should become one cluster.

16. CLI Reference

Key-value commands

Command	Description
<code>set KEY VALUE</code>	Replicated write; forwarded to the leader if needed
<code>get KEY</code>	Local read from the applied KV state
<code>delete KEY</code>	Replicated delete; forwarded to the leader if needed

Cluster inspection

Command	Description
<code>status</code>	Local role, term, leader, log length and commit index
<code>members</code>	Current stable or joint membership configuration
<code>log [LIMIT]</code>	Local Raft log, optionally limited to the last entries

Cluster management

Command	Description
<code>addnode HOST:PORT</code>	Add a node ID/address to membership
<code>rmnode HOST:PORT</code>	Remove a node ID/address from membership
<code>join HOST:PORT</code>	Join the target cluster
<code>merge HOST:PORT</code>	Merge this cluster with another cluster
<code>stop/start</code>	Stop or resume local Raft processing
<code>quit</code>	Stop the process

17. REST API

The same process exposes HTTP endpoints on `--bind`. Client-facing endpoints are useful for simple scripts; Raft protocol endpoints are used by peer nodes.

Client and inspection endpoints

Endpoint	Method	Purpose
<code>/status</code>	GET	Local node summary
<code>/cluster/config</code>	GET	Membership configuration
<code>/log?limit=N</code>	GET	Local log entries
<code>/kv/<KEY></code>	GET	Local KV read
<code>/kv/set</code>	POST	Replicated <code>set</code> ; body <code>{"key": "...", "value": "..."}"</code>
<code>/kv/delete</code>	POST	Replicated delete; body <code>{"key": "..."}"</code>

Endpoint	Method	Purpose
<code>/cluster/add-node</code>	POST	Add a node; body <code>{"node": "HOST:PORT"}</code>
<code>/cluster/remove-node</code>	POST	Remove a node; body <code>{"node": "HOST:PORT"}</code>
<code>/cluster/join</code>	POST	Add caller node to the cluster
<code>/cluster/membership-request</code>	POST	Request a target member list
<code>/cluster/merge</code>	POST	Merge membership with another cluster
<code>/node/stop</code>	POST	Stop local Raft processing
<code>/node/start</code>	POST	Resume local Raft processing

Raft transport endpoints

Endpoint	Method	Purpose
<code>/raft/message</code>	POST	Incoming Raft protocol message
<code>/raft/client-command</code>	POST	Forwarded client command

For example:

```
1 curl -s http://127.0.0.1:7400/status
2 curl -s -X POST http://127.0.0.1:7400/kv/set \
3   -H 'content-type: application/json' \
4   -d '{"key": "color", "value": "blue"}'
5 curl -s http://127.0.0.1:7401/kv/color
```

18. Demo Troubleshooting

The joining node stays alone

- Check that `--join` points to a running node.
- Check that both processes can reach each other's `--bind` addresses.
- Run `members` and `log` on the introducer to see whether the membership entries were committed.

Writes work on one node but reads look stale elsewhere

- `get KEY` is a local read. Wait a few ticks or inspect `status` to confirm the follower has applied the committed entry.
- Use the leader for read-after-write demos when you need immediate feedback.

A restarted node has unexpected old data

- Check the default data directory: `var/demo/<host>_<port>`.
 - Use a fresh `--data-dir` for isolated experiments.
-

Part III - Reference

19. Public API Reference

Package Exports

```
1 from raft_rx import (
2     ClusterConfiguration,
3     Command,
4     JsonFileStorage,
5     JsonlTelemetrySink,
6     LogEntry,
7     ManualClock,
8     MemoryTransport,
9     Message,
10    MessageKind,
11    NodeConfig,
12    NullTelemetrySink,
13    RaftApplication,
14    RaftCluster,
15    RaftNode,
16    RaftShell,
17    Role,
18    SystemClock,
19 )
```

NodeConfig

Field	Type	Description
<code>node_id</code>	<code>str</code>	Unique node identifier
<code>peers</code>	<code>list[str]</code>	Other nodes the node can talk to

Field	Type	Description
<code>election_timeout_ms</code>	<code>int</code>	Base election timeout
<code>heartbeat_interval_ms</code>	<code>int</code>	Leader heartbeat interval; default 50
<code>initial_members</code>	<code>list[str] \ None</code>	Explicit initial membership; defaults to node plus peers

RaftCluster

Method	Description
<code>add_node(config, paths, application, period_us=0)</code>	Add a node and register its FSMs
<code>provision_node(node_id, members, running=False, election_timeout_ms=450)</code>	Create a node during reconfiguration
<code>tick(advance_ms=10)</code>	Advance one runtime tick
<code>run(ticks, advance_ms=10)</code>	Advance several ticks
<code>leader()</code>	Return the current leader or <code>None</code>
<code>submit_to_leader(command)</code>	Submit to the current leader if one exists
<code>summaries()</code>	Return per-node status dictionaries
<code>stop_node(node_id) / start_node(node_id) / restart_node(node_id)</code>	Lifecycle helpers
<code>export_trace(path) / report_trace(path) / serve_trace(...)</code>	Trace helpers

RaftNode

Method / property	Description
<code>role</code>	Current <code>Role</code>
<code>submit_command(command)</code>	Enqueue a client command for this node
<code>request_membership_change(members)</code>	Request full target membership
<code>summary()</code>	Return a status dictionary
<code>last_committed_command()</code>	Return the last applied command if available
<code>stop() / start()</code>	Local node lifecycle

20. Building and Testing

Run the library tests:

```
1 uv run --project python --extra dev pytest -q
```

Run the standalone demo tests:

```
1 uv run --project python/examples/demo_app --extra dev pytest -q
```

Build the standalone demo package:

```
1 cd python/examples/demo_app
2 uv build
```

21. Troubleshooting

No leader is elected

- Ensure every node has the same initial membership.
- Use staggered election timeouts in deterministic examples.
- Advance enough simulated time with `cluster.run(...)`.
- Check whether nodes were stopped through `stop_node` or `node.stop()`.

Commands are not committed

- Submit writes to the leader, or use a layer that forwards to the leader.
- Run enough ticks after submission.
- Check `status`, `leader`, and `log NODE` in the shell.

A joining node does not participate

- In in-memory clusters, call `cluster.provision_node(...)` before requesting the membership change.
- Pass the full target member list to `request_membership_change`.
- Run until the cluster returns to stable membership.

Snapshot restore misses state

- Ensure `snapshot()` returns the complete application state.
- Ensure the returned object is JSON-serializable when using `JsonFileStorage`.
- Keep all state mutation inside `apply` and `restore_snapshot`.